User Testing Guide: user_testing.py

How to drive the RAG-Pipe server by hand from a Windows command line, and what each example teaches about how retrieval-augmented generation works.

1. What user_testing.py Is

Picture the RAG server as a librarian who keeps a box of index cards. Every time you *add* a document, the librarian cuts it into index-card-sized chunks and files each one. When you *search*, she doesn't read anything out loud — she just hands you the cards that are closest in meaning to what you asked. When you *ask*, she does both: pulls the closest matching cards, actually reads them, and answers your question using only what's written on those cards.

user_testing.py is you, standing at that counter. It's a small interactive command-line program — type a command, see what the librarian does, type another. Unlike the project's automated test scripts (which run a fixed list of checks and tell you pass/fail), this one is for exploring: add whatever you want, ask whatever you want, and watch the raw scores and sources come back so you can build your own sense of how the retrieval step and the answering step actually behave.

2. Starting It

The RAG server has to already be running in Docker — user_testing.py is just a client that talks to it over HTTP, the same way a web browser talks to a website.

```
C:\...\RAG-Pipe> docker compose up -d
C:\...\RAG-Pipe> python user_testing.py
RAG-Pipe user testing -- talking to http://127.0.0.1:8000
{'status': 'ok', 'documents_stored': 0}

Commands:
  add <text>      add a document (it gets chunked + embedded server-side)
  search <query>  show the stored chunks closest to a query, with scores
  ask <query>     ask a question; the model answers from retrieved chunks
  status          show server health and how many chunks are stored
  clear           wipe the in-memory index
  help            show this message
  quit / exit     leave

>
```

That first status line is a free health check: if the server is up, you see documents_stored; if it's not running yet, you get a connection error instead of a confusing crash later.

3. Command Reference

Command	What it does
`add <text>`	Chunks and embeds the text, adds it to the in-memory index.
`search <query>`	Pure retrieval — ranks stored chunks by cosine similarity to the query. No language model involved.

`ask <query>`	Retrieves the relevant chunks, then has the language model answer using only those chunks as context.
`status`	Health check: is the server up, how many chunks are stored.
`clear`	Empties the index. There's no database behind this — it's the same as what happens on a container restart.
`help / quit`	Show the command list, or leave.

4. Worked Examples (Real Output, Captured Live)

Every transcript below is real output from a live run against the Docker containers, not a mock-up. Run them yourself in the same order to see the identical behavior.

Example 1: Health check on an empty index

Right after docker compose up -d, before adding anything:

```
> status
{'status': 'ok', 'documents_stored': 0}
```

What this shows: *The index is just a Python list in memory. It starts empty every time the container starts, and there's no database file sitting around from a previous run.*

Example 2: Add one document

```
> add Penguins are birds that cannot fly. They swim instead, and an
  emperor penguin can stand over a metre tall.
{'message': 'added', 'total_documents': 1}
```

What this shows: *add immediately chunks and embeds the text server-side (you don't see the chunking here because this text is short enough to be one chunk) and reports the running total.*

Example 3: Search -- retrieval with no language model

```
> search how tall can a penguin get
  1. [0.7443] penguins are birds that cannot fly. they swim instead, and
     an emperor penguin can stand over a metre tall.
```

What this shows: *search never calls the language model at all -- it's pure vector math. The number in brackets is a cosine-similarity score (0 to 1): how close the question's meaning is to the chunk's meaning, even though the words 'how tall' don't literally appear in the chunk.*

Example 4: Ask -- the language model answers from that chunk

```
> ask how tall can a penguin get
  thinking...

  Answer: According to the context, an emperor penguin can stand over a
  metre tall.

  Sources:
    1. [0.7443] penguins are birds that cannot fly. they swim instead,
       and an emperor penguin can stand over a metre tall.
```

What this shows: *ask does what search does, then feeds the matching chunk(s) to the language model as its only source of truth and asks it to answer from them. The 'Sources' list is not decoration -- it's exactly which chunks the model was given, so you can check its work.*

Example 5: Ask something the index can't answer

Same index as above (still just the one penguin document):

```
> ask what is the capital of France
  thinking...

  Answer: I don't know -- nothing in the indexed documents looks
  relevant to this question.

  Sources: (none -- nothing in the index cleared the relevance floor)
```

What this shows: *A well-behaved RAG server should refuse rather than guess when nothing relevant is indexed. This server enforces that with a minimum similarity score (0.3) -- if no stored chunk clears that bar, it won't even call the language model, so there's no chance of it hallucinating an answer about France from a document about penguins.*

Example 6: Ask the same question after adding unrelated documents

Adding two documents that have nothing to do with penguins, then re-asking:

```
> add The Eiffel Tower is a wrought-iron lattice tower in Paris,
  France, completed in 1889.
{'message': 'added', 'total_documents': 2}
> add Mount Everest is the tallest mountain above sea level, located
  in the Himalayas.
{'message': 'added', 'total_documents': 3}
> ask how tall can a penguin get
  thinking...

  Answer: According to the context, an emperor penguin can stand over a
  metre tall.

  Sources:
    1. [0.7443] penguins are birds that cannot fly. they swim instead,
       and an emperor penguin can stand over a metre tall.
```

What this shows: *Even with the Eiffel Tower and Mount Everest sitting in the same index, neither shows up as a source and the answer doesn't change. Retrieval is a ranking problem: the embedding model keeps meanings far enough apart that unrelated noise in the index doesn't dilute a clear match.*

Example 7: A question that needs two chunks combined (multi-hop)

Adding two related paragraphs about metal detectors, then asking a question neither paragraph answers alone:

```
> add Very Low Frequency (VLF) detectors ... typically between 3 kHz
  and 30 kHz ... Pulse Induction (PI) detectors ... far less affected
  by mineralized or salty ground ...
{'message': 'added', 'total_documents': 4}
> add Ground mineralization is one of the biggest challenges ... Soils
  with a high iron oxide content ... mask the signal from a real
  target ...
{'message': 'added', 'total_documents': 5}
> ask Why would someone hunting for gold in heavily mineralized soil
  prefer a pulse induction detector over a VLF detector?
  thinking...

  Answer: ... pulse induction detectors are less affected by
  mineralized or salty ground, which can generate their own magnetic
  response that can mask the signal from a real target, while VLF
  detectors are more affected by this ground noise or chatter.

  Sources:
    1. [0.6910] very low frequency (vlf) detectors ...
    2. [0.5532] ground mineralization is one of the biggest challenges ...
```

What this shows: *Two different paragraphs both clear the relevance floor, so both get handed to the model, and the answer draws on facts from each. This is what 'multi-hop' retrieval looks like: no single stored chunk has the full answer, but the question's meaning is close enough to both that retrieval pulls them together.*

Example 8: Clear the index

```
> clear
{'message': 'cleared'}
> status
{'status': 'ok', 'documents_stored': 0}
> search how tall can a penguin get
(no results -- index is empty?)
```

What this shows: *clear wipes the in-memory list immediately -- the next search or ask has nothing to retrieve from, exactly like right after a fresh docker compose up -d.*

5. What To Watch For

The score in brackets is the whole retrieval story. A high score (0.6+) means the embedding model thinks the chunk and the question mean similar things. A low score means it doesn't, no matter how many keywords overlap.

An empty Sources list is a feature, not a bug. It means the relevance floor did its job and the server chose to say "I don't know" instead of forcing in a weak match.

Try asking the same question with and without the language model (search vs. ask) on the same index -- it's the clearest way to see where retrieval ends and generation begins.

To test the public tunnel the same way, set BASE_URL before launching: set BASE_URL=https://your-tunnel.trycloudflare.com then python user_testing.py.

Big Takeaway

user_testing.py turns three invisible steps -- chunk/embed, rank by similarity, and generate from only the top matches -- into something you can trigger one command at a time and actually see. The eight examples above cover the whole pipeline: storing (add), pure retrieval (search), grounded generation (ask), honest refusal when nothing matches, resistance to irrelevant noise in the index, combining two sources for one answer, and resetting back to empty.